

Quick Start Guide: Building an Academic Website

This is a guide meant to help set up and manage an academic website like gking.harvard.edu.

What We Use (and Why)

Tool	What it is	Why we use it
Hugo	An open-source program that turns plain text files into a functional website (including styling, urls, etc).	Hugo has a number of advantages. Firstly, it produces very fast pages because it is a static site generator (as opposed to website builders like WordPress, which construct pages from scratch every time a person visits them, Hugo sites are built as a static entity, like a brochure, and are just handed to the person trying to access them with no personalization). As a result, security risks are also much lower compared to a dynamic website framework (hackers would normally target live servers or databases, which don't exist for static sites—Hugo sites are read-only and pre-compiled). The simpler nature of rendering static sites also means that hosting a Hugo site is free.
Hugo Blox	In order to make website design easier, Hugo has a number of pre-made themes, and Hugo Blox is the academic website theme.	Built into this theme are all the website elements that an academic website would normally need, like our publication pages, people profiles, and search bar. It's easier to use this structure than to start from scratch.
Tailwind CSS	A styling system that handles the aesthetics of the website	To avoid writing custom design code, Tailwind makes

		the site look modern and aesthetically consistent.
Pagefind	A search engine that runs in the browser of the person visiting the site, instead of on the website's server, as would be the case with dynamic websites.	Static sites have many advantages, but one tradeoff is that because there is no server running in the background, a normal search function wouldn't work. Instead, Pagefind creates an index (like what would be at the back of a reference book), and ships that as part of the website: when someone types something in the search bar of the website, a program running in their own browser scans that index and shows results without any server necessary.
Github	A version-control system where the site's files live	GitHub is just like a shared drive for the website: it tracks changes, makes those changes quickly reversible, and allows people to easily collaborate on a project like a website.
Github Pages	Free website hosting from GitHub that takes our folder of website files and makes them visible to people on the internet	Once files are in the Github repository, Github serves them at a public url. This is a free way to host a website and is commonly used alongside Hugo.
Github Actions	A digital helper in Github that automates repetitive tasks like automatically running Hugo/Pagefind/etc when you push a change to the website, and deploying the result to Github Pages.	Without Github Actions, the process of making updates would be long and tedious and require manually rebuilding the site on a local device with every change and uploading the result every time.
Claude (in Cursor)	The LLM that makes website creation/editing accessible to non-technical users	In this particular case, Claude's main job is to translate English into code

		(you say “I want to upload this pdf to the website to accompany x paper”, and Claude turns that into code that is understandable to the relevant tools above).
Cursor	A desktop app that has Claude built in. It is a code editor like VSCode, but you won’t need to actually edit any code yourself.	Cursor is where you talk to Claude. Using Claude in Cursor allows you to build the website locally on your computer and push changes to GitHub all in one place.
HTML	The language web browsers understand, that describes the structure of a web page (every page on the internet is HTML). Hugo generates HTML automatically from the plain text files we edit in GitHub.	HTML is non-negotiable in terms of website-building, but you’ll never have to write it yourself.

What You’ll Need for Website Setup

1. The ACADEMIC_SITE_PROMPT.md document: this is a detailed document that describes to Claude exactly how the website should look and function (it is essentially the website blueprint).
2. The website’s content as laid out in the prompt document (this can either be migrated by Claude from an existing academic website, or uploaded in the form of a CV, bio, publication and presentation pdfs, etc).

Implementation

1. Download and install [Cursor](#). This is the only application you need on your computer. Cursor is a code editor with Claude built in — it’s where you’ll talk to Claude and where Claude will do all its work.

During setup, Cursor will ask you to sign in or create an account. Choose a plan that gives you access to Claude (the Pro plan or above).

2. Navigate to <https://github.com/> and create an account. In the home page of your github account, you should see a plus icon (+). Click on it and select “new repository.” Fill out the form so it looks like this, but with a name of your choosing:

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1

General

Owner *

KatalinaToth

Repository name *

new-site

new-site is available.

Great repository names are short and memorable. How about [fuzzy-barnacle](#)?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

3. Open the `ACADEMIC_SITE_PROMPT.md` document. Fill out the “Who I am” section. You’ll see that it asks you to provide either the url of a preexisting academic website, or a series of documents and pieces of information. Provide as much as you can up front to reduce back and forth; Claude will ask you for anything else that it needs.

4. Tell Claude:
"Create a new academic website for [Name], using the website building instructions attached [attach completed website prompt PDF]. The GitHub repository I'd like to use is at [link to the repository you just created]. Install Hugo so that we can do local builds in future."

Claude will walk you through the authentication process so that it can connect to GitHub.

5. Claude will scaffold the site, create the GitHub repository, and deploy it. If there are changes to be made (it is likely that you'll want to customize the website to some extent), you can ask Claude to make the changes you want. For common changes, you can refer to the prompts below.
6. If you're making changes to the site after it's live online, it is wise to build it locally (so you can see the changes you're making on your own device without immediately putting them on the internet) before pushing changes to the repository. In order to do this, tell Claude “I want to try making X and X changes, but build them locally first and give me a

localhost link so I can approve them before we push.” Claude should give you a localhost link (something like “localhost:1313”) that you can click on to view the website before committing any edits to the internet. Once you’re satisfied, say “commit and push to main”. If multiple people have access to/are editing the site you’re working on, tell Claude to “pull any recent changes to main” before you start working to make sure you don’t end up pushing changes that conflict with changes someone else has made.

How It Works

1. You tell Claude what you want.
2. Claude edits the relevant text files.
3. Claude pushes the change to GitHub.
4. GitHub Actions automatically rebuilds the site.
5. The live site updates.

If Something Goes Wrong

- A new change isn’t visible on the site: Check GitHub Actions (repository → Actions tab). If the latest run has a red X, tell Claude: "The build failed. Here's the error: [paste]." If it’s a yellow circle, it’s still working and you’ll just need to wait a while longer. If there is no related update in the Actions tab, it’s possible that Claude didn’t commit and push. Ask Claude to make sure it pushed, and if it already did to check that it pushed to the right branch of the repository.
- A new change doesn’t look good: Everything is versioned. Tell Claude "revert the last change" or "go back to how the site was yesterday."

Sample Claude Prompts

To add a new journal article:

“Add a new journal article to *name of your website repository*.

- Title: XXX
- Authors: XXX (comma-separated)
- Publication venue: XXX (e.g. "American Political Science Review, 111, 3, Pp. 484–501")
- Year: XXX
- Abstract: XXX
- Publisher's DOI or URL: XXX
- Dataverse DOI (if any): XXX
- Topic: XXX (pick one of the research areas Claude has generated/you have created for your site homepage)

The article PDF is attached. Commit and push.”

To customize URLs for specific pages/papers:

“Add a new short URL redirect to *name of your website repository*.

- Short URL path: XXX (this becomes your equivalent of gking.harvard.edu/XXX)
- Destination: XXX (full URL or site path like /publication/quest/)

Commit and push.”